Programming Languages Project 2

Athina Bampzeli

May 2025

1 Question 1

Using the dataset you have selected, a) split the data into training and test sets, and b) choose at least three classification or regression models (depending on the type of data and excluding neural networks) from the scikit-learn library (or other libraries) and apply them to the data. Report the prediction accuracy on both the training and test sets and justify the hyperparameter selection process. If you have not selected your own dataset, choose a different dataset from the one used in the previous assignment.

I chose to work with the dataset "BCFactorDataset". I separated the data to train and test sets as can be seen in Figure 1. As for the regression models, I tried LinearRegression, DecisionTreeRegressor, RandomForestRegressor and KNeighborsRegressor. The code can be seen on my GitHub account.

```
#split data into train and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, shuffle=True)

print(f"Training set X size: {X_train.shape[0]}, Test set X size: {X_test.shape[0]}")
print(f"Training target y size: {y_train.shape[0]}, Test target y size: {y_test.shape[0]}")

print("Available features: ", dataset.list_features())
print("Available targets: ", dataset.list_targets())
```

```
Training set X size: 640, Test set X size: 160
Training target y size: 640, Test target y size: 160
Available features:  ['NumAromaticRings', 'NumHAcceptors', 'NumHeteroatoms', 'NumRotatableBonds', 'NumVale
Available targets:  ['bcf']
```

Figure 1: Train and test set

Overfitting occurs when a machine learning model learns the details and noise of the training data too well. As a result, it performs exceptionally well in the training data, but poorly in new unseen data. This happens when the model is too complex and the model memorizes the training data instead of learning general patterns. Underfitting occurs when a model is too simple and does not learn the underlying patterns in the training data. This leads to poor performance on both the training and test data because the model lacks the complexity needed to capture the relationships between variables. Overfitting and underfitting are generally considered opposite ends of the bias-variance tradeoff in machine learning, so they cannot occur at the same time for the same model on the same dataset. The Performance Metrics used are Mean Squared Error, R^2, Mean Absolute Error and Mean Absolute Performance Error.

Linear Regression

The LinearRegression results suggest mild underfitting, as indicated by relatively low R^2 values (0.546 in the training and 0.467 in the test set), meaning the model explains less than half of the variance. The small gap between train and test metrics (MSE, MAE, and R^2) suggests the model generalizes reasonably well and is not overfitting. However, the extremely large MAPE values are likely due to near-zero true values, making this metric unreliable in this context. The performance metrics regarding the train and test sets can be seen in Table 1.

Table 1: LinearRegression Performance Metrics

	MSE	R^2	MAE	MAPE
Train set	0.82	0.55	0.74	2.5E+13
Test set	0.85	0.47	0.68	4.8E+13

Decision Tree Regression

The DecisionTreeRegressor results indicate clear signs of overfitting. The model achieves near-perfect performance on the training set (R^2 = 0.9991, MSE = 0), but test performance drops significantly (R^2 = 0.5550, MSE = 0.7128), showing that the model has memorized the training data rather than learning general patterns. Performance metrics before and after tuning the hyperparameters can be seen in Table 2. To reduce overfitting, the model limited the maximum depth of the tree and increased the minimum samples per leaf, while allowing splits with fewer samples to maintain flexibility. The best hyperparameter values that were calculated via GridSearchCV can be seen in Table 3. The relatively close values of R^2 and MSE in the training and test set indicate a better generalization and a reduction in overfit compared to the previous model.

Table 2: DecisionTreeRegressor Performance Metrics

	Train MSE	Train R^2	Test MSE	Test R^2
No tuning	0.002	1	0.71	0.56
Hyperparameter tuning	0.44	0.76	0.52	0.68

Table 3: DecisionTreeRegressor best Hyperparameters

Max Depth	Min Samples Leaf	Min Samples Split
5	4	2

To evaluate model performance, a learning curve is plotted (Figure 2) to assess how training size impacts R^2 scores and reveal potential underfitting or overfitting. The training score remains consistently high, close to 1.0, while the validation score increases at first but then plateaus, indicating overfitting. The significant gap between training and validation performance suggests the model fits the training data very well but struggles to generalize to unseen data. In Figure 3 a heatmap of GridSearchCV results visualizes the effect of different hyperparameter combinations, specifically maximum depth and minimum samples per split, on cross-validation R^2 performance. The heatmap highlights a region where performance is optimal, indicating that moderate values for depth and sample split yield better generalization.

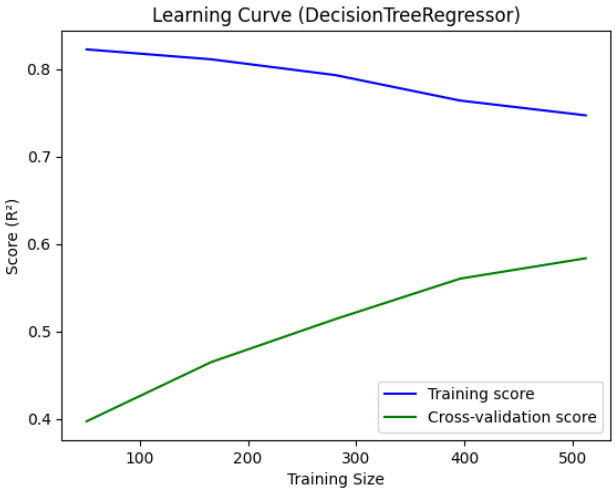

Figure 2: DecisionTreeRegressor Learning Curve

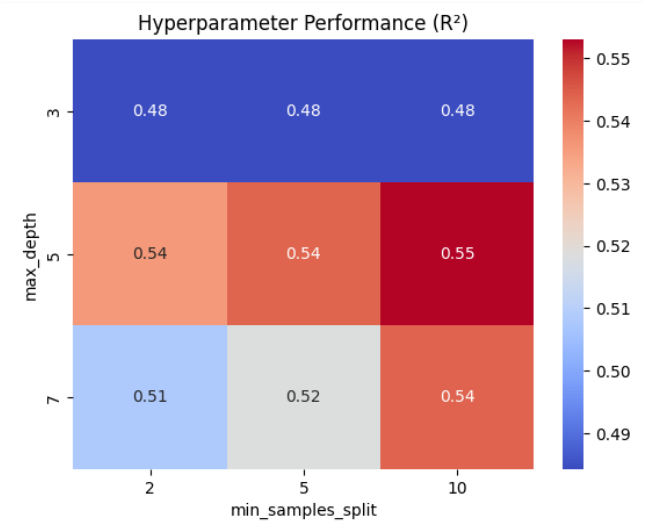

Figure 3: DecisionTreeRegressor Heatmap

Random Forest Regression

The RandomForestRegressor results indicate strong overall performance with good generalization. The model achieves a high R^2 on both the training set (0.9026) and test set (0.7869), with a moderate gap between them, suggesting only mild overfitting. The relatively low MSE and MAE on both sets further support that the model captures the underlying patterns well. However, the unusually high MAPE values again suggest the presence of near-zero actual values, making MAPE an unreliable metric. Performance metrics can be seen in Table 4. The best hyperparameter values that were calculated via GridSearchCV can be seen in Table 5. The optimized model achieves a very high training R^2 of 0.9579 and a strong test R^2 of 0.7911. The gap between training and test MSE indicates a higher model complexity and slight overfitting, but generalization remains good. Performance metrics after tuning the hyperparameters can be seen in Table 6.

Table 4: RandomForestRegressor Performance Metrics before tuning

	MSE	R^2	MAE	MAPE
Train set	0.18	0.90	0.30	7.9E+12
Test set	0.34	0.79	0.43	2.8E+13

Table 5: RandomForestRegressor best Hyperparameters

n estimators	Min Samples Leaf	Min Samples Split
200	1	2

Table 6: RandomForestRegressor Performance Metrics after tuning

	Train MSE	Train R^2	Test MSE	Test R^2
Hyperparameter tuning	0.08	0.96	0.33	0.79

A learning curve is plotted for the RandomForestRegressor (Figure 4) to evaluate how R^2 scores change with increasing training set size. The training and validation curves converge to high values with only a small gap between them, indicating strong generalization and minimal overfitting. This suggests that the model benefits from more data and is capable of learning meaningful patterns without memorizing the training set. A heatmap of GridSearchCV results (Figure 5) visualizes the impact of maximum depth and minimum samples per split (with fixed number of estimators) on cross-validation performance, aiding in the identification of optimal hyperparameter combinations (maximum depth=7, minimum samples per split=2).

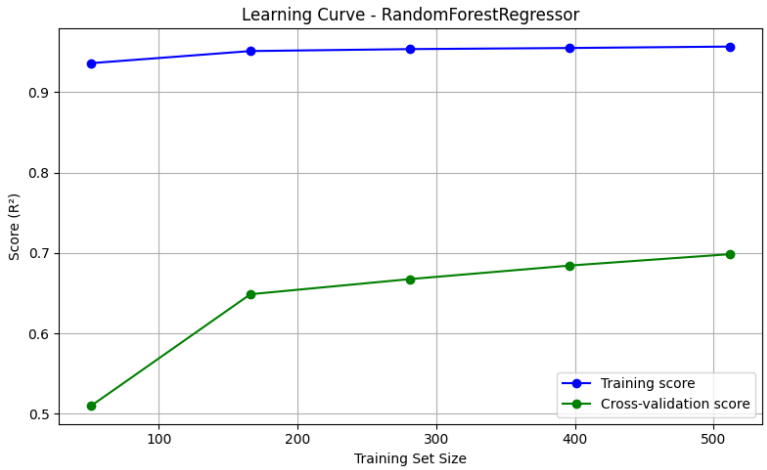

Figure 4: RandomForestRegressor Learning Curve

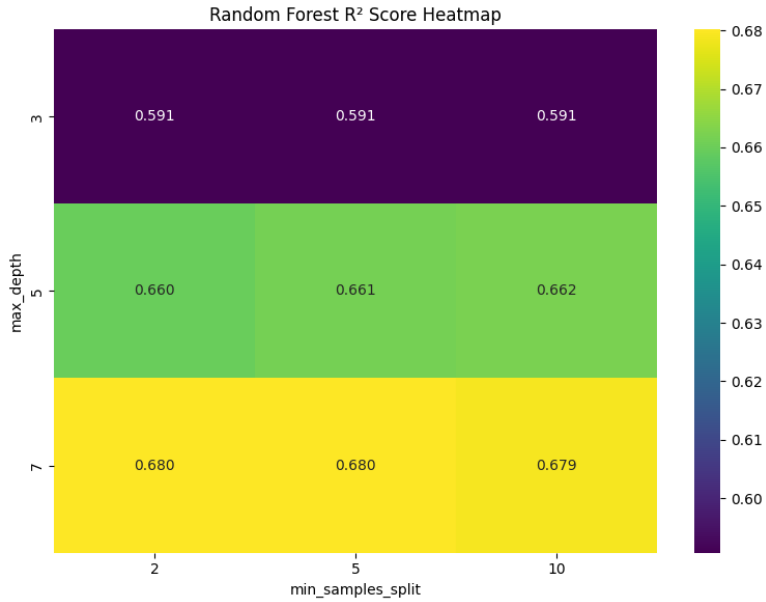

Figure 5: RandomForestRegressor Heatmap

K Neighbors Regression

The KNeighborsRegressor results indicate moderate underfitting in the initial model. The R^2 scores are 0.7541 in the training set and 0.6704 in the test set, with relatively close MSE values (0.4458 and 0.5280 respectively), suggesting limited model complexity and reasonable generalization. However, after hyperparameter tuning, the model displayed signs of overfitting. The optimized model achieved near-perfect training performance (R^2 = 0.9991, MSE = 0.0016), while test performance improved only modestly (R^2 = 0.7020, MSE = 0.4774), indicating the model learned training patterns too specifically. Performance metrics before and after tuning can be seen in Table 7. The best hyperparameter values identified via GridSearchCV are shown in Table 8. These settings (increased number of nearest neighbors, small leaf size, brute algorithm) increased model complexity, improving training accuracy but slightly compromising generalization.

Table 7: KNeighborsRegressor Performance Metrics

	Train MSE	Train R^2	Test MSE	Test R^2
No tuning	0.45	0.75	0.53	0.67
Hyperparameter tuning	0.002	1	0.48	0.70

Table 8: KNeighborsRegressor best Hyperparameters

Leaf Size	n Neighbors	Algorithm
20	10	Brute

A learning curve is plotted for the KNeighborsRegressor (Figure 6) to assess model performance across varying training sizes, revealing overfitting through a large gap between training and validation R^2 scores. Additionally, a heatmap of GridSearchCV results (Figure 7) filtered for weights='uniform' and algorithm='auto', illustrates how different combinations of number of nearest neighbors and leaf size influence validation performance. Based on the heatmap the leaf size doesn't influence the R^2 as much as the number of nearest neighbors does.

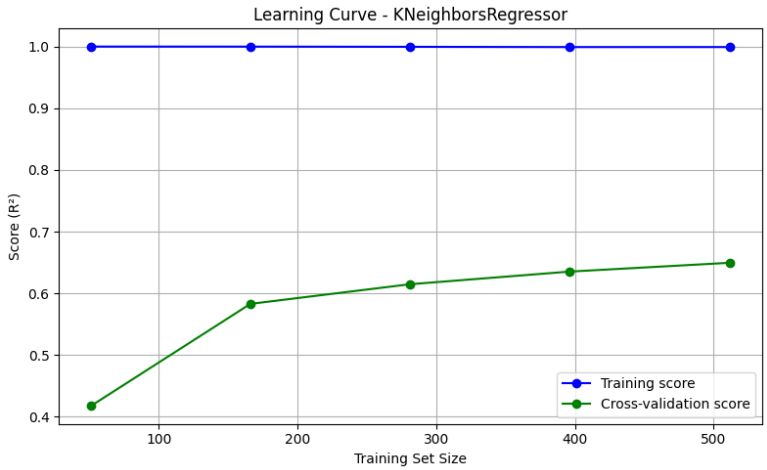

Figure 6: KNeighborsRegressor Learning Curve

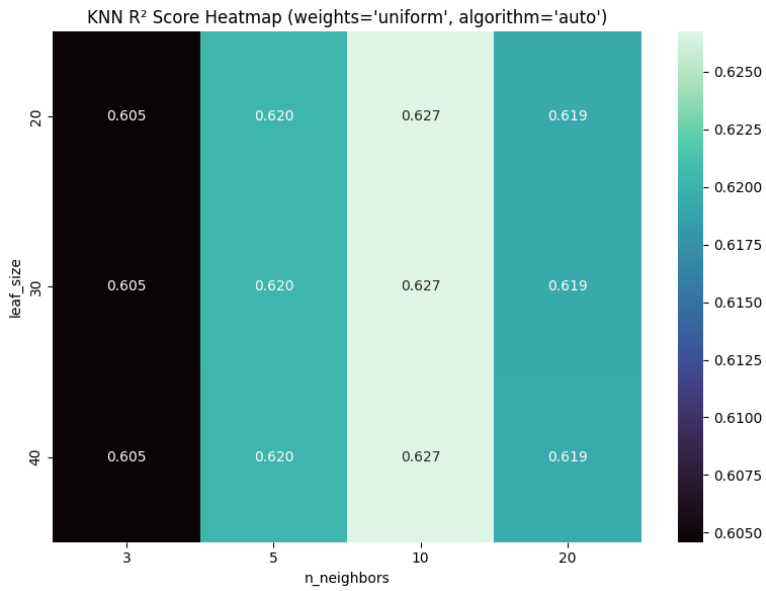

Figure 7: KNeighborsRegressor Heatmap

2 Question 2

Develop at least three different neural network architectures and train the corresponding models on the selected dataset. Report the prediction accuracy on both the training and test sets and justify the hyperparameter selection process.

Three different neural network architectures were developed and trained on the selected dataset, with the goal of predicting the bioconcentration factor.

Model Descriptions

- **Model 1: Simple MLP**
  This model includes a single hidden layer with 64 neurons and ReLU activation, followed by a linear output layer. It serves as a basic reference architecture.

- **Model 2: Deeper Network with Dropout**
  A deeper architecture consisting of three hidden layers (128, 64, 32 neurons) with ReLU activations and a dropout layer (rate = 0.3) for regularization. This model was designed to improve generalization and reduce overfitting.

- **Model 3: Smaller Network with `tanh`**
  This model uses two hidden layers with 32 and 16 neurons respectively, and `tanh` activations. It was designed to test a smaller and simpler architecture using a different activation function.

Training and Evaluation

All three neural network models were trained using the Adam optimizer, with Mean Squared Error (MSE) as the loss function and Mean Absolute Error (MAE) as a secondary performance metric. Both inputs and targets were normalized before training, and predictions were inverse-transformed to the original scale for evaluation. The performance metrics for each model are presented in Table 9. The extremely high MAPE values indicate potential issues with outliers or target values near zero thus making MAPE an unreliable metric as mentioned in Section 1.

Model 2, the deeper architecture with dropout regularization, achieved the best performance overall, with a test MAE of 0.53 and a test R^2 of 0.71, indicating good generalization and learning capacity. Model 1, the simple MLP, showed consistent performance on both training and test sets, suggesting a balanced tradeoff between model complexity and generalization. Model 3, a smaller network with tanh activations, exhibited the weakest performance with a test MAE of 0.67 and R^2 of 0.61, likely due to limited model capacity and activation function choice.

Table 9: Neural Network Performance Metrics

	Train MAE	Test MAE	Train R^2	Test R^2
Model 1	0.49	0.57	0.77	0.68
Model 2	0.44	0.53	0.80	0.71
Model 3	0.61	0.67	0.67	0.61

The training and validation loss curves illustrate the model's convergence behavior over epochs, showing how the MSE decreases during training (Figure 8, Figure 10, Figure 12). The MAE plots provide additional insight into the model's prediction accuracy on both training and validation datasets throughout the learning process (Figure 9, Figure 11, Figure 13). For Model 1, the MSE and MAE curves decrease smoothly and in parallel between training and validation, indicating steady convergence and minimal overfitting. Model 2 shows more pronounced fluctuations in the validation loss and MAE curves compared to the training ones, pointing to slight instability, but it still achieves the best overall accuracy. The curves of Model 3 decrease more slowly and plateau at higher error levels, reinforcing the conclusion that it underfits the data. The small model size and use of tanh activations may limit its ability to capture complex patterns in the data.

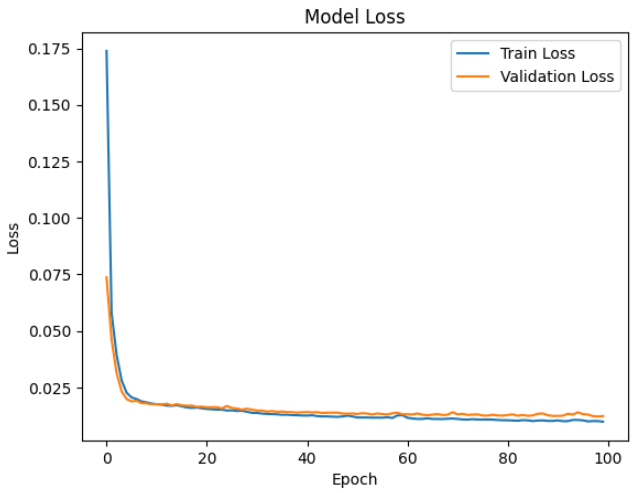

Figure 8: Model 1 Loss Curves

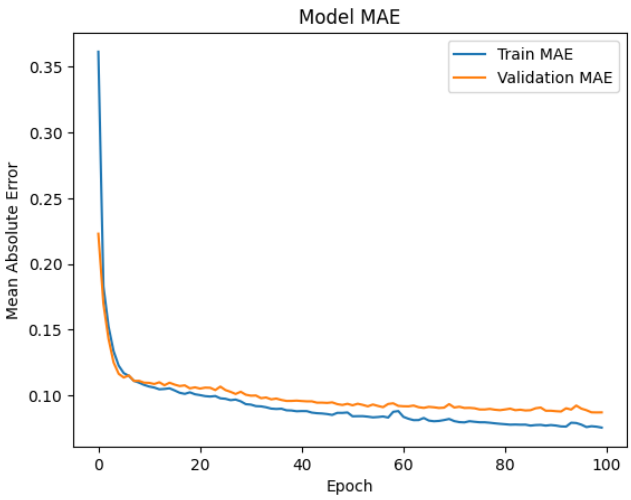

Figure 9: Model 1 MAE

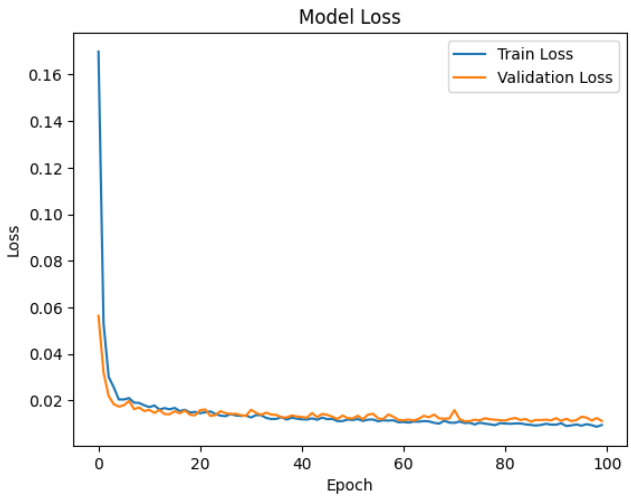

Figure 10: Model 2 Loss Curves

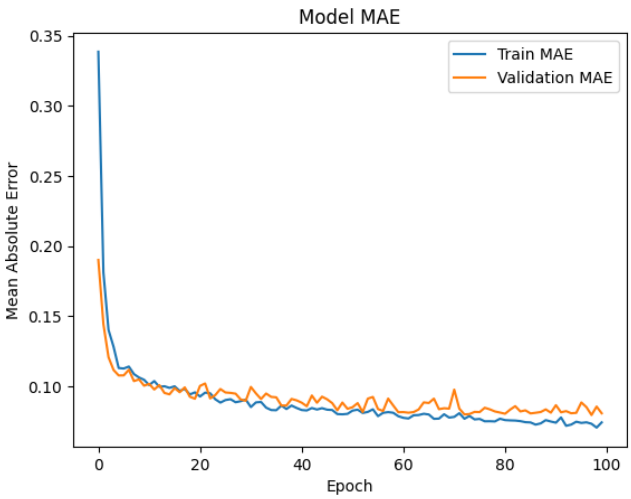

Figure 11: Model 2 MAE

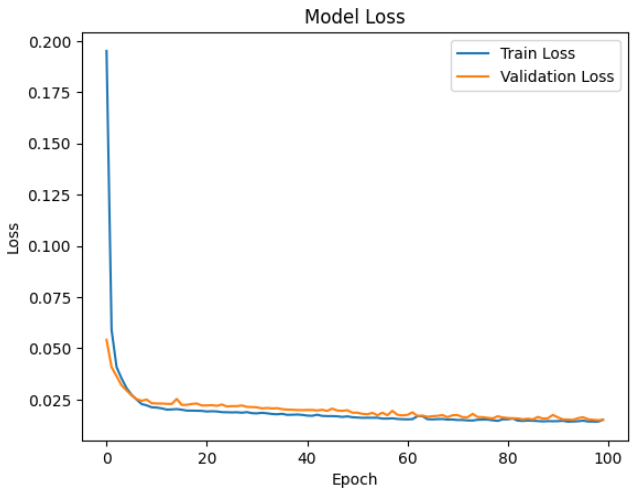

Figure 12: Model 3 Loss Curves

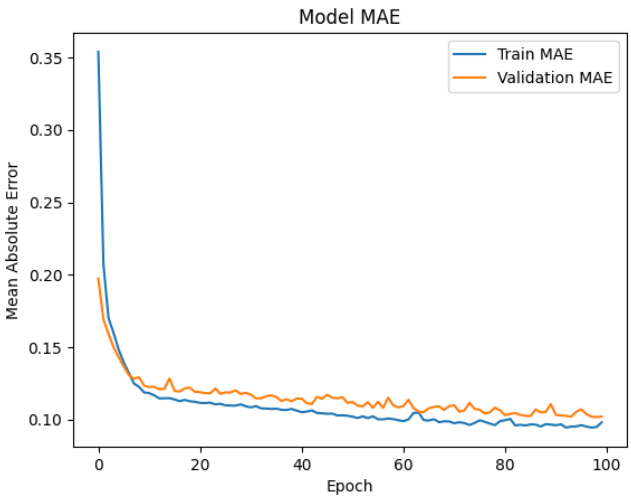

Figure 13: Model 3 MAE

Hyperparameter Selection

The hyperparameters, such as the number of layers and number of neurons can be optimized after multiple experiments and comparisons. The following trial-and-error process helps identify settings that improve learning

while avoiding overfitting. ReLU was the primary activation function due to its effectiveness in non-linear learning, while `tanh` was tested in Model 3 for comparative purposes. The dropout rate of 0.3 was selected to improve generalization without severely limiting learning. In Table 10, Table 11 and Table 12 the performance metrics for different values of the hyperparameters number of layers/neurons of the 3 models can be seen.

For Model 1, increasing the number of layers from 1 to 2 improved training R^2 from 0.77 to 0.82, but test R^2 slightly decreased, indicating potential overfitting (Table 10). The performance of Model 2 was improved by selecting the configuration 128, 32, 32 which achieved the best test R^2 of 0.76, outperforming the original architecture (Table 11). In contrast, Model 3 showed minimal sensitivity to hyperparameter variations across different configurations, the performance remained poor, reinforcing that a small model with tanh activations is insufficient for this problem (Table 12). These results suggest that deeper networks with adequate regularization and larger hidden layers tend to generalize better, while overly simplistic models fail to capture the complexity of bioconcentration factor prediction

Table 10: Performance Metrics for Hyperparameter Selection of Model 1

	Train MAE	Test MAE	Train R^2	Test R^2
64 neurons/1 layer	0.49	0.57	0.77	0.68
80 neurons/1 layer	0.49	0.57	0.77	0.69
64, 64 neurons/2 layers	0.43	0.57	0.82	0.67

Table 11: Performance Metrics for Hyperparameter Selection of Model 2

	Train MAE	Test MAE	Train R^2	Test R^2
128, 64, 32 neurons/3 layers	0.44	0.53	0.80	0.71
64, 64, 32 neurons/3 layers	0.50	0.58	0.77	0.67
128, 32, 32 neurons/3 layers	0.43	0.52	0.81	0.76

Table 12: Performance Metrics for Hyperparameter Selection of Model 3

	Train MAE	Test MAE	Train R^2	Test R^2
32, 16 neurons/2 layers	0.61	0.67	0.67	0.61
16, 16 neurons/2 layers	0.61	0.67	0.67	0.61
32 neurons/1 layer	0.67	0.73	0.61	0.52

3 Question 3

Reduce the dimensionality of the data using at least two different dimensionality reduction algorithms and visualize the results. You should display different classes or regression targets using different colors (if feasible). Perform clustering on the data and comment on the results.

Dimensionality reduction was performed using two different algorithms: PCA and t-SNE, as can be seen in Figure 14. Both methods reduced the normalized training data to two components, allowing for easy visualization. The PCA plot reveals the main directions of variance in the data, while t-SNE provides a non-linear embedding that better captures local data structure. The target variable is represented with a color gradient to highlight the distribution of values across the reduced dimensions. In both plots, the color gradient shows that the reduced features still preserve useful information, as some clustering or gradient patterns are visible.

KMeans clustering with 3 clusters was applied to the PCA-transformed data (Figure 15). The resulting clusters appear partially aligned with regions of similar target values, indicating that the PCA transformation retained enough structure for meaningful grouping. However, there is some overlap between clusters, suggesting that three clusters may not fully separate the data or that the target variable is not strictly cluster-based.

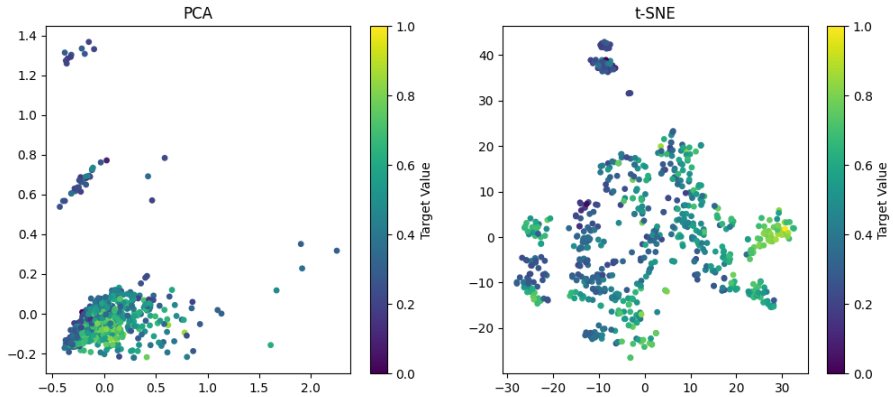

Figure 14: Dimensionality Reduction

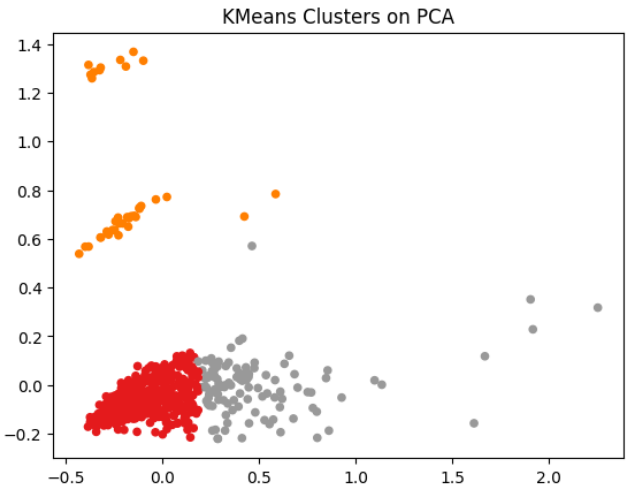

Figure 15: KMeans Clustering

4 Question 4

Develop a surrogate model using your dataset and use it to optimize one input parameter of your model. Report the validation results of the surrogate model, as well as the results obtained using different optimization methods.

A surrogate model was created to approximate the relationship between the input features and the target output. The surrogate model was trained using a Random Forest Regressor and achieved good predictive performance, with a validation R^2 of 0.80 and a low mean squared error of 0.00675, indicating it captures the underlying trends in the bioconcentration factor (BCF) data effectively. By varying the first input feature while keeping others fixed at their mean values, the model predicted a minimum BCF of 0.4353 at a normalized input of 0.0000. A more refined continuous optimization using `scipy.optimize.minimize_scalar` identified an optimal input value of 0.2361, corresponding to a slightly higher predicted BCF of 0.4909. This small discrepancy suggests the presence of local variation in the surrogate surface, which is expected given the non-smooth behavior of Random Forest models. Overall, the results demonstrate that the surrogate model can be used to explore input conditions that minimize BCF, supporting safer chemical design. The results can be seen in Figure 16.

```
Validation MSE: 0.0067511195823428865
Validation R²: 0.8004402281673505
Best input value for feature 0: 0.0000 with predicted BCF: 0.4353
Optimal input (feature 0): 0.2361
Minimum predicted BCF: 0.4909
```

Figure 16: Surrogate Model Results